Supplementary Material

1 Estimating the Latest Invalidation Time

While exact invalidation times can be recorded with a 4-byte per-block timestamp, the memory overhead is prohibitive. We therefore adopt an age-counter scheme [6] that stores a compact per-block counter and estimates invalidation time from that counter. Concretely, we maintain a 1-byte counter per block and increment it while the block remains valid, so that upon invalidation the counter approximates the write-to-invalidation interval (i.e., the block's invalidation time).

Counter update rule. We increment each block's counter on every GC relocation of that block; the key question is by how much. To measure elapsed time without per-block timestamps, we keep a 4-byte per-segment timestamp that tracks a segment's residency progress. Using the timestamp, we compute the segment age by (now - segment's timestamp), and add the age to the block's counter. Considering the counter's limited capacity, we quantize time at a granularity of 130K and increment the counter accordingly; that is, the counter increases by one for every 130K blocks (the logical time unit is defined in §2.1). This per-segment accounting enables the counter to reflect the time spent in each segment, resulting in a more accurate estimate of the latest invalidation time.

Range considerations. With 130K quantization and a 1-byte counter, the representable span is approximately 33M blocks of logical time. For a hot block handled in HotFilter, the BIR upper bound is lower than 33M, so range is not an issue. When the ML model predicts a non-hot block, there can be extremely cold blocks whose residency exceeds 33M blocks. However, such cases can be separated using other features. Another set of features used for the ML model (frequency and recency-weighted frequency) can represent values up to 320M, which is sufficiently large to characterize cold blocks. Accordingly, we preserve accuracy by combining complementary features that capture hot and cold blocks [5, 8]. In our experiments, this design yields only 1.4% average and 2% maximum drop in accuracy compared to a model that uses a 4-byte latest invalidation time.

Therefore, using a 1-byte per-block counter, combined with a 4-byte per-segment timestamp, consumes approximately 32MiB for a 128GiB system, reducing the memory footprint by 4× compared to a 4-byte per-block timestamp.

2 Detailed Analysis for Candidate Features

To design an effective ML model for predicting invalidation time, we first analyze eleven candidate features. These features are derived from prior work [1, 3, 4, 7, 9] and are summarized in Table 1. For each feature, we report two measures: the Pearson correlation coefficient (PCC) and mutual information (MI). PCC shows the degree of linear correlation between the feature and the actual invalidation time, while MI primarily measures non-linear dependency. Using both ensures that we capture features that are strongly correlated as well as those that contain hidden, non-linear information [2]. Although including all features could potentially increase the information available to the model, it would also enlarge the model complexity and allow less informative features to act as noise. This trade-off motivates selecting a compact yet effective subset of features.

Table 1: PCC and MI scores of all candidate features. Bold values indicate the features selected in ML-Alloc.

Feature	PCC score	MI score
LBA	**0.410**	**0.799**
LBA (uphalf)	0.181	0.379
LBA (downhalf)	0.011	0.017
Previous accessed LBA (near)	0.335	**0.615**
Previous accessed LBA (far)	0.095	0.179
Frequency	**0.657**	0.376
Frequency per chunk	**0.586**	0.476
Recency-weighted frequency	**0.727**	0.371
Latest invalidation time	**0.616**	0.425
Average invalidation time	0.445	0.515
Sequentiality	0.055	0.023

LBA represents the logical block address of the data. It has a very high MI score, showing that the address space itself contains important information. Two partitioned forms of LBA (uphalf and downhalf) are also tested, but they show low scores. We also check the previously accessed LBA. Here, *near* means the LBA of the block written immediately before the current one, while *far* refers to the LBA of the block written ten updates earlier. The near case shows high MI, meaning it can capture sequential or clustered access patterns, whereas the far case provides little information because its relation to the current access is weaker.

Frequency-related features are also important. The frequency of updates to a block within a window shows high correlation with invalidation time, because hot blocks tend to become invalid quickly. Frequency per chunk generalizes this to a spatial region, capturing shared hotness within a 2MB chunk. Recency-weighted frequency gives more weight to recent updates and achieves the highest correlation score, making it a strong signal.

The latest invalidation time also shows a high correlation: if a block is updated very recently, it is likely to be invalidated soon. We also test average invalidation time, which smooths the update pattern and sequentiality, which detects sequential writes, but these two have lower scores compared to others.

In summary, we select six features that provide the balance of predictive power and efficiency. Two features (LBA and previously accessed LBA (near)) are chosen for their high MI scores, capturing non-linear and contextual information about access locality. Four others (frequency, frequency per chunk, recency-weighted frequency, and latest invalidation time) are chosen for their strong PCC scores, reflecting direct correlation with invalidation time. Together, this subset offers a compact representation that retains the most informative signals while avoiding unnecessary complexity and noise from weaker features. This careful selection enables *ML-Alloc* to achieve accurate predictions with manageable model cost.

References

[1] Chandranil Chakraborttii and Heiner Litz. Reducing Write Amplification in Flash by Death-Time Prediction of Logical Block Addresses. In *Proceedings of the International Conference on Systems and Storage*, pages 1–12, 2021.

[2] Huanhuan Gong, Yanying Li, Jiaoni Zhang, Baoshuang Zhang, and Xialin Wang. A New Filter Feature Selection Algorithm for Classification Task by Ensembling Pearson Correlation Coefficient and Mutual Information. *Engineering Applications of Artificial Intelligence*, 131:107865, 2024.

[3] Jeong-Uk Kang, Jeeseok Hyun, Hyunjoo Maeng, and Sangyeun Cho. The Multi-Streamed Solid-State Drive. In *Proceedings of the USENIX Workshop on Hot Topics in Storage and File Systems*, page 13, 2014.

[4] Seonggyun Oh, Jeeyun Kim, Soyoung Han, Jaeho Kim, Sungjin Lee, and Sam H Noh. MIDAS: Minimizing Write Amplification in Log-Structured Systems through Adaptive Group Number and Size Configuration. In *Proceedings of the USENIX Conference on File and Storage Technologies*, pages 259–275, 2024.

[5] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on pattern analysis and machine intelligence*, 27(8):1226–1238, 2005.

[6] Subhash Sethumurugan, Jieming Yin, and John Sartori. Designing a Cost-Effective Cache Replacement Policy Using Machine Learning. In *Proceedings of the International Symposium on High-Performance Computer Architecture*, pages 291–303, 2021.

[7] Penghao Sun, Litong You, Shengan Zheng, Wanru Zhang, Ruoyan Ma, Jie Yang, Guanzhong Wang, Feng Zhu, Shu Li, and Linpeng Huang. Learning-Based Data Separation for Write Amplification Reduction in Solid State Drives. In *Proceedings of the Design Automation Conference*, pages 1–6, 2023.

[8] Jorge R Vergara and Pablo A Estévez. A review of feature selection methods based on mutual information. *Neural computing and applications*, 24(1):175–186, 2014.

[9] Qiuping Wang, Jinhong Li, Patrick PC Lee, Tao Ouyang, Chao Shi, and Lilong Huang. Separating Data via Block Invalidation Time Inference for Write Amplification Reduction in Log-Structured Storage. In *Proceedings of the USENIX Conference on File and Storage Technologies*, pages 429–444, 2022.